OpenPsi

1. Introduction

OpenPsi is a computational model of emotion proposed by Cai, Goertzel, Zhou and colleagues (2013), built on PSI theory, a psychological model of motivation and behavior originally described informally by German psychologist Dietrich Dorner. It also borrows implementation ideas from Joscha Bach's MicroPsi.

The core idea sets it apart from rule based models like OCC: emotion is not a separate module driven by fixed appraisal rules. Instead, the agent is an autonomous system driven by internal demands (physiological, cognitive, and social), and emotion emerges from the dynamics of the whole system as perception, cognition, and action selection interact.

To test the model, the authors used it to control a virtual robot in a Minecraft style world across three scenarios, then analyzed the resulting emotional behavior using Lewis's dynamic theory of emotion, which frames an emotional episode as three phases: trigger, self-amplification, and self-stabilization.

Emotions Matter in Games

Video games are everywhere, and what makes one *succeed* is immersion, the state where the game world is convincing enough that the player forgets they're playing and loses themselves in it. The authors lean on Nicole Lazzaro's claim that emotions are the key to great experiences. So the logic chain is: good games need immersion, immersion needs believable characters, and believable characters need emotions. That sets up the whole reason for building an emotion model.

The Emotion Research And The Gaps

The Research says that the main most effective computing research falls into 2 areas First one is ***Emotion Recognition*** and second one is ***Modelling The Causes of Those Emotions*** and what is the rare in the research on the effects of emotions, which they break into four threads

1. formal models of cognitive appraisal theory (how an event gets evaluated into an emotion),
2. how emotion influences cognition,
3. how multiple emotional states interact during an agent's reasoning,
4. **Emergent Emotions**, meaning emotion that arises from a simple agent interacting with its environment rather than being scripted.

What PSI Theory?

Most emotion models work like a vending machine. Something happens, you check a rulebook, and out pops the matching emotion. "Player insulted the character" goes in, "anger" comes out. The OCC model works exactly like this: a big list of if-this-then-feel-that rules. The emotion is a separate part you can look up on demand.

PSI says no, that's not how feelings actually work. There's no emotion box sitting off to the side waiting to be queried.

Instead, picture the agent as a creature with **needs**. It needs energy, it needs to feel safe, it needs to feel competent, maybe it needs company. When one of those needs slips out of balance, it creates a kind of internal pressure (an urge), and that pressure pushes the agent to act. Meanwhile the agent is also perceiving its world, thinking, and choosing what to do, all at the same time.

Emotion is what shows up when all of that runs together. It's not produced by a rule. It's a byproduct of the whole machine being in a certain state. Low energy and being blocked from food might leave the agent in what we'd recognize as "anger," but nobody wrote a rule that said "feel anger now." It just emerged from the situation.

So, the one-line difference:

I. **OCC:** event → rulebook → emotion (looked up)

II. **PSI:** needs + perception + thinking + action all interacting → emotion (emerges on its own)

Lewis's Dynamic View of Emotion

Lewis agrees with Dorner that emotion is **non-linear and dynamic**, an emergent phenomenon of the whole system. His specific contribution is to describe an emotion episode as a sequence of **phase transitions**:

I. **Trigger phase**: a perturbation interrupts the system's order.
II. **Self-amplification phase**: positive feedback loops between perception, emotion, and attention make small deviations rapidly grow.
III. **Self-stabilization phase**: negative feedback eventually overtakes the amplification, change slows, and the system settles into a stable state.

2. The Emotional Model Of PSI Theory

The defining feature of PSI is how it handles **why an agent does anything at all**. PSI's answer: every goal-directed action traces back to a **motive**, and every motive traces back to an **urge**, which is just a demand the agent has. Those demands come in three flavors:

I. **physiological** (energy, water, staying intact),

II. **cognitive** (wanting to feel certain about the world, wanting to feel competent),

III. **social** (wanting connection with other agents).

So the chain runs: a demand exists → it creates an urge → the urge gives rise to a motive → the motive drives an action. Nothing the agent does is arbitrary or top-down commanded. It's all pushed from underneath by needs.

Then the historical bit: to prove PSI actually works and isn't just a theory on paper, Dorner built real virtual agents controlled by it. These agents were **little steam vehicles** living in a simulated world, and they survived by managing **fuel and water**. That's the concrete demo of the idea, an agent whose entire behavior is organized around keeping its basic demands satisfied.

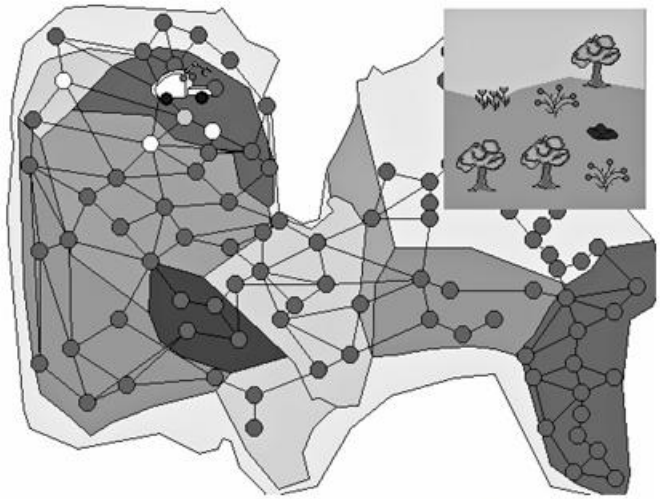


Fig. 1. The PSI agent and the island, adopted from Dörner (2003).

A PSI agent has no "executive structure," meaning there's no central controller or manager module deciding what to do. Nothing sits at the top issuing orders. The agent runs purely on demands pushing from below. This is the same anti-module idea from before, applied to control itself.

\

The three kinds of demands, with concrete examples this time:

I. **External resource demands:** water, energy, and integrity (staying undamaged). These tie to physical things in the world.

II. **Abstract cognitive demands:** certainty (wanting to understand the world, reduce surprise) and competence (wanting to feel capable). These aren't about objects, they're about the agent's internal sense of how things are going.

III. **Social demand:** affiliation, the need for connection. The key detail is that this one *can only be satisfied by other agents*, not by anything the agent does alone.

Thresholds turn demands into urges. Each demand has a target threshold, a healthy level. When the demand drifts away from that threshold, that gap (the "deviation") gets felt as an **urge**. The bigger the gap, the stronger the urge. The urge then becomes an **intention** or **motive**, something the agent wants to act on.

Many motives, one ruler. At any moment several demands might be off-target, so several motives can exist at once. But the system doesn't try to chase all of them. A single **"ruling motive"** takes over and dominates. The agent commits to one thing rather than thrashing between many.

How the ruler gets picked. The winning motive is chosen by combining two factors:

I. **strength of the urge** (how badly off the demand is), and

II. **estimated chance of realization** (how likely the agent thinks it can actually fix it).

So it's not just "whatever hurts most wins." A desperate but hopeless need can lose to a milder need the agent can actually do something about. Roughly, it's urgency weighed against feasibility.

Then action follows. Once the ruling motive is set, the agent produces actions aimed at satisfying it.

The Loop

A PSI agent roughly follows a "sense, think, act" cycle, but here's the crucial twist: these three don't happen one after another in a clean sequence. They run as **parallel processes** that are constantly influencing each other. The agent isn't waiting to finish perceiving before it starts planning, and it isn't waiting to finish planning before it acts. Everything is happening at once and tugging on everything else.

Where Action Comes From

Every action is born from a **motivational impulse**, and those impulses come from the **predefined dynamic demands.**

"**Predefined**" means the *set* of demands is fixed (energy, integrity, certainty, etc.), but "**dynamic**" means their levels are constantly changing as the agent lives. So the demands are the engine that produces all behavior.

Modulators

Perception, memory retrieval, and action-control are all shaped by a small set of tuning parameters called **modulators**. The big conceptual move here is this line: the combined setting of these modulators *is* what we interpret as the agent's **emotional configuration**.

Earlier the paper said emotion isn't a separate module. This is the payoff of that claim. Emotion isn't stored anywhere, it's just *the current values of the modulators*. Change how the agent perceives, remembers, and acts, and you've changed its emotion. They're the same thing viewed two ways.

The Four Modulators (Dorner's set)

Activation

the agent's readiness to perceive and react. It's a dial between two modes of operation. High activation means fast, intense, reactive behavior, like being keyed up. Low activation means slow, reflective, thoughtful behavior. So activation governs the tradeoff between acting quickly and thinking carefully.

Resolution level

the *accuracy* or thoroughness of the agent's thinking (perception, planning, action regulation). High resolution means careful, detailed processing. Low resolution means sloppy, quick-and-dirty processing. The key relationship: it **drops as activation rises**. They're inversely linked. The example makes it intuitive: an angry agent has high activation, so its resolution crashes, and it stops thinking about the consequences of its actions. This is the model's way of capturing "acting without thinking." Worked up emotionally means less careful cognitively.

Securing Threshold

controls how often the agent runs **securing behavior**, which is basically *checking the environment for unexpected changes* (scanning for surprises, threats, things that shifted).

Two things drive it:

I. It's proportional to the strength of the current motive. When the agent is in urgent pursuit of something, the threshold goes high, which means less checking. The logic: if you're desperate and focused, you don't waste time looking around, you just go. (Note the inverse wording: high threshold = less securing behavior.)
II. It also depends on certainty. In an uncertain, unpredictable environment, the threshold goes low, meaning more checking. The logic: if you don't know what's going on, you'd better keep scanning.

Selection Threshold

how *resistant* the agent is to switching between competing motives. Remember there can be multiple motives at once, only one ruling. This modulator protects the current ruler. A high selection threshold means the agent sticks with what it's doing and won't get distracted; a low one means it switches easily. Its main job is to **prevent oscillation**, the agent flip-flopping uselessly between two goals. It **rises with activation**. The example: when fleeing a threat (high activation), the agent locks hard onto its integrity (survival) demand and won't get pulled off it, that's high selection threshold doing its job.

How An Emotion Is Actually Represented In The Model

Emotions are regions in a space, not labels. Instead of storing "anger" or "sadness" as named things, the model builds a multidimensional space out of a few **proto-emotional dimensions**, and an emotion is just a *region* (an area) sitting somewhere in that space. The dimensions themselves come from the basic demands and the modulators we've already covered. So emotion is literally a location in modulator-space.

The Six Dimensions

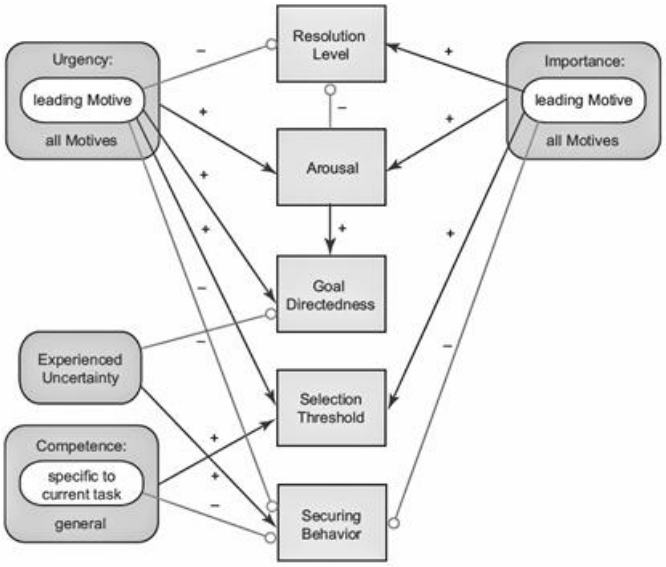
I. Pleasure
II. Activation
III. Resolution level
IV. Securing threshold
V. Selection threshold
VI. Level of goal-directed behaviour

Emotions

regions, with examples. Once you have the space, every named emotion is just a characteristic region in it:

- **Sadness:** low activation, high resolution, weak motive dominance, low goal-orientedness. Picture this: slowed down, overthinking, no strong drive pulling you, not really pursuing anything. That reads as sad.
- **Anger:** high activation, low resolution, weak motive dominance, low goal-orientedness. Worked up, not thinking clearly, no single motive firmly in charge, and not productively pursuing a goal. That reads as angry.

Notice sadness and anger share two coordinates (weak motive dominance, low goal-orientedness) and differ on the other two (activation and resolution are flipped). That's exactly the point of using a continuous space: emotions are neighbors and overlaps, not isolated boxes.



OCC is the textbook approach, 22 emotions, each triggered by specific conditions involving events, actions, and objects. It's popular precisely because it's easy to program. But it's still a rulebook, which is the thing OpenPsi is built to avoid.

emotion in PSI is **not an isolated component**. It emerges from the dynamics of the whole system, where perception, cognition, and action selection, all controlled by the modulators, interact together. This is the now-familiar core claim, placed right next to OCC so the difference is unmissable. OCC = rules producing emotion. PSI = system dynamics producing emotion.

Joscha Bach took the original PSI theory, extended it, and built a concrete working model called **MicroPsi**.

The reasoning behind that choice: by *not* hardwiring the rest of the cognitive stack, OpenPsi keeps the freedom to plug in whatever modern AI techniques you want for those parts later. MicroPsi commits to PSI's way of doing everything; OpenPsi commits only to PSI's emotion engine and leaves the rest as open slots.

3. Lewis's Dynamics Of An Emotional Model

Lewis says it's a **loop**: your appraisal shapes your emotion, *and* your emotion shapes how you appraise. They feed each other. That's why he calls them **"appraisal-emotion amalgams"**, the two are fused together, not a clean cause-then-effect line.

Appraisal = the mind's evaluation of a situation (judging whether it's good or bad, threatening or safe, for you), which is what produces an emotion.

Lewis views emotion as a **self-organizing process**, not a fixed value. He calls appraisal and emotion **"amalgams,"** meaning they are fused in a feedback loop: appraisal shapes emotion and emotion shapes appraisal, so each is both cause and effect of the other.

An emotional episode unfolds as a **phase transition** in three stages:

I. **Trigger** — a perturbation disrupts the system's order and raises its sensitivity.

II. **Self-amplification** — perception, emotion, and attention lock into a positive feedback loop, and small deviations grow rapidly.

III. **Self-stabilization** — negative feedback eventually overtakes the amplification, change slows, and the system settles into a coherent stable state.

Lewis names this completed arc an **Emotion Interpretation (EI)**.

The model rests on two dynamic-systems assumptions from Smith and Thelen (2003):

I. **Multicausality** — order emerges from the interaction of many parts with no central controller (self-organization), the same principle behind PSI's "no executive structure" agent.

II. **Nested timescales** — emotional change happens at multiple stacked speeds: an *emotional episode* (seconds to minutes) sits inside a *mood* (hours to days), which sits inside *personality* (years). Faster layers are nested within slower ones, keeping the system coherent across time.

This dynamic-systems framing is the theoretical lens the paper later uses to interpret the robot's emotion graphs in the experiments.

Lewis's Model: The Four-Part Crux

Lewis's dynamical-systems approach bridges the **psychological** and **neurobiological** accounts of emotion, letting emotion be described as both a mental and a physical process. His model has four parts.

1. Trigger phase — The system is running in an orderly state until a perturbation interrupts it. Order is rapidly lost and sensitivity to the environment spikes. The system reorganizes into a new pattern, so the trigger itself *is* the phase transition.

2. Self-amplification phase — Positive feedback dominates. The system is highly sensitive, so small deviations are rapidly amplified. This is the runaway stage where the emotion builds on itself.

3. Self-stabilization phase — Negative feedback takes over, change slows, and continuity rises. The system settles into a coherent state. This stable base enables *complexification*, allowing the appraisal to become more elaborate and detailed.

4. Learning — Recurring patterns get encoded as biases, beliefs, traits, and emotional habits. Stabilization is the precondition for learning: once appraisals stabilize, their interpretations and action plans persist, and learning occurs by strengthening the connections among elements that activate together (a Hebbian "fire together, wire together" mechanism).

Mapping to the two assumptions:

I. The phase transitions arise from **multicausality** (many processes interacting with no central controller).

II. Coherence across **nested timescales** (episode → mood → personality) is achieved through **learning**, which links a fleeting emotional episode to lasting moods and personality.

Table 1
Scales of emotional development (Lewis, 2000).

Attribute	Emotional episode	Mood	Personality
Timescale	Seconds, minutes	Hours, days	Years
Description	Rapid convergence of a cognitive interpretation with an emotional state	Lasting entrainment of interpretative bias with a narrow emotional range	Lasting interpretive-emotional habits specific to classes of situations
Dynamic system formalism	Attractor	Temporary modification of state space	Permanent structure of state space
Possible neurobiological mechanism	Cortical coherence mediated by orbito-frontal organization entrained with limbic circuits	Orbitofrontal-corticolimbic entrainment, motor rehearsal, and preafference, sustainded neurohormone release	Selection and strengthening of some corticocortical and corticolimbic connections, pruning of others, loss of plasticity
Higher-order form	Intention, goal	Intentional orientation	Sense of self

4. OpenPsi

4.1. Architecture overview

The OpenPsi is implemented within the framework of Open Cog, which is an open-source Artificial General Intelligence (AGI) software under active development. OpenCog has been used in the area of natural language processing (Lian et al., 2010), data mining, and controlling virtual dogs in virtual worlds (Goertzel et al., 2010)

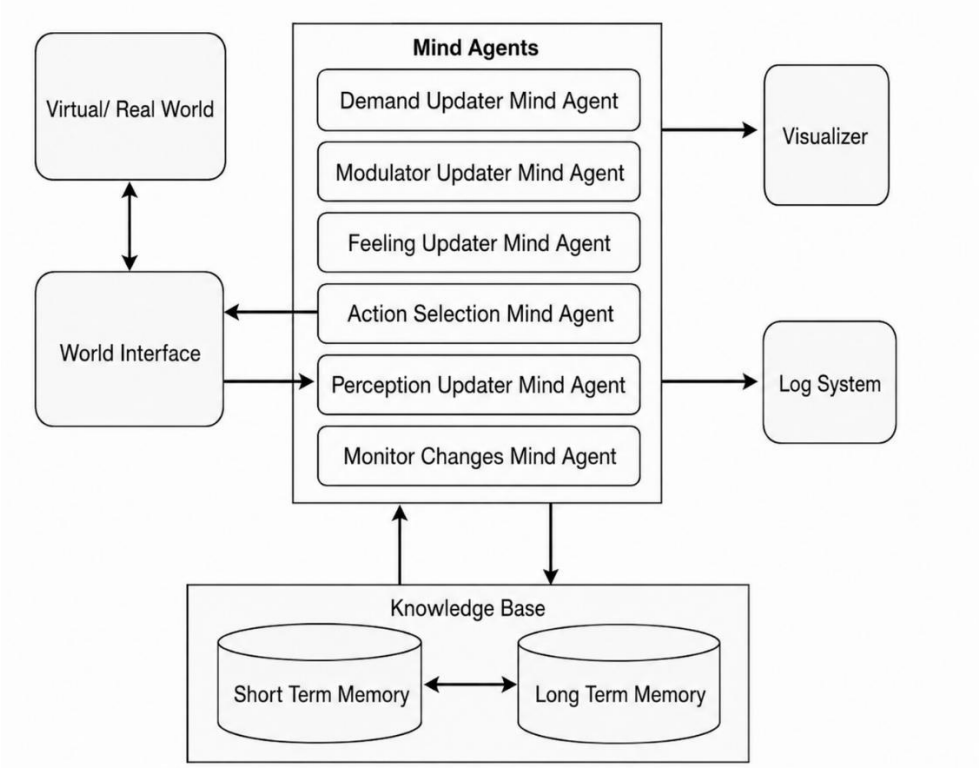
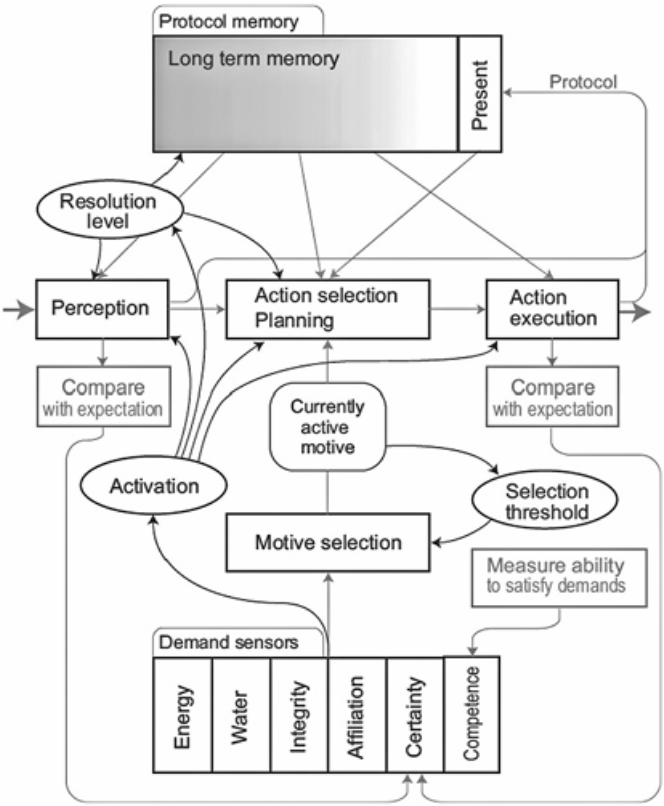


Fig. 4. Framework overview.

The general architecture of OpenPsi is presented in Fig. 4. It consists of six ‘‘mind agents’’ running on top of the knowledge base. A ‘‘mind agent’’ is a software object that could access the knowledge base and runs concurrently like system thread. Since each ‘‘mind agent’’ is relatively simple and does very specific jobs, none can induce complex behaviors on itself. However, all the ‘‘mind agents’’ are closely related via the global knowledge base and talk with each other through messages, which may result in more interesting dynamics. The inherent emergence of this architecture fits the principle of PSI theory quite well, that is emotions emerge from the dynamics of the whole system.

Regarding to ‘‘World Interface’’, it is similar to ‘‘world adapter’’ in MicroPsi (Bach, 2003; Bach et al., 2006). It provides a generic interface between OpenPsi and outside world, which can be a game world or even the real world in future. It converts data coming from the world into the format desired by Perception UpdaterMindAgent, and encapsulates actions generated by ActionSelectionMindAgent into requests sent back to the outside world.



4.2. Knowledge Representation

What the AtomSpace is. It's the interface for storing and manipulating Atoms (the edges/vertices from the hypergraph). Structurally it follows a **client-server model**:

- **one AtomSpace Server**, and
- **many clients**, one owned by each mind agent.

Where Atoms actually live. The Atoms aren't held in the server itself, they're stored in a **back-end database**, which can be either a relational database (like SQL) or a NoSQL one (the paper mentions Redis elsewhere). The server's job is to be the middleman: it takes requests from the mind agents and **retrieves or stores Atoms** in that database on their behalf.

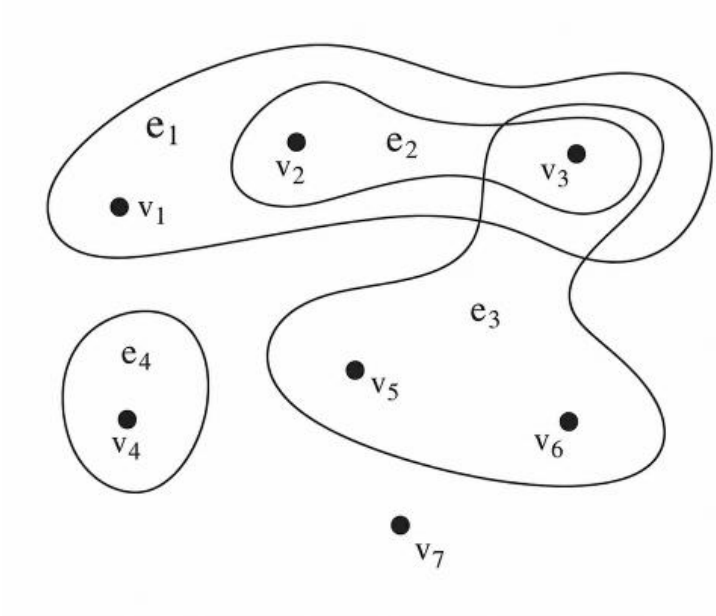
How communication works. Server and clients talk by **messages**, in a simple request-response pattern: a client sends a request, the server does the work against the database, and sends the result back. So no agent touches the database directly, everything goes through the server.

The mind agent's point of view (this is the clever part). From inside a mind agent, the workflow is just: pull some Atoms, do its computation, push results back, all through its own client. And here's the important design consequence, the mind agent is deliberately kept *ignorant* of two things:

- it doesn't know the **back-end database** exists (the server hides it), and
- it doesn't even know the **other mind agents** exist, as long as it doesn't need to explicitly message them.

Why that matters. This is abstraction and decoupling done on purpose. Each agent only sees "I read and write the knowledge base through my client." It doesn't care what kind of database is underneath, or that five other agents are working at the same time. This is exactly what makes the system distributable and scalable, you could swap the database, move agents to different machines, or add agents, without any single agent needing to know.

There's also a subtle but important point hiding here: the agents are decoupled but **not independent**. They never talk to each other directly, yet they deeply influence one another, because they're all reading from and writing to the *same shared knowledge base*. One agent's changes to the Atoms become another agent's input. So the coordination happens *through the shared data*, not through direct conversation. That's how "simple agents, no central boss, emergent complexity" actually works in practice.



Advantages of the AtomSpace / Hypergraph knowledge Representation

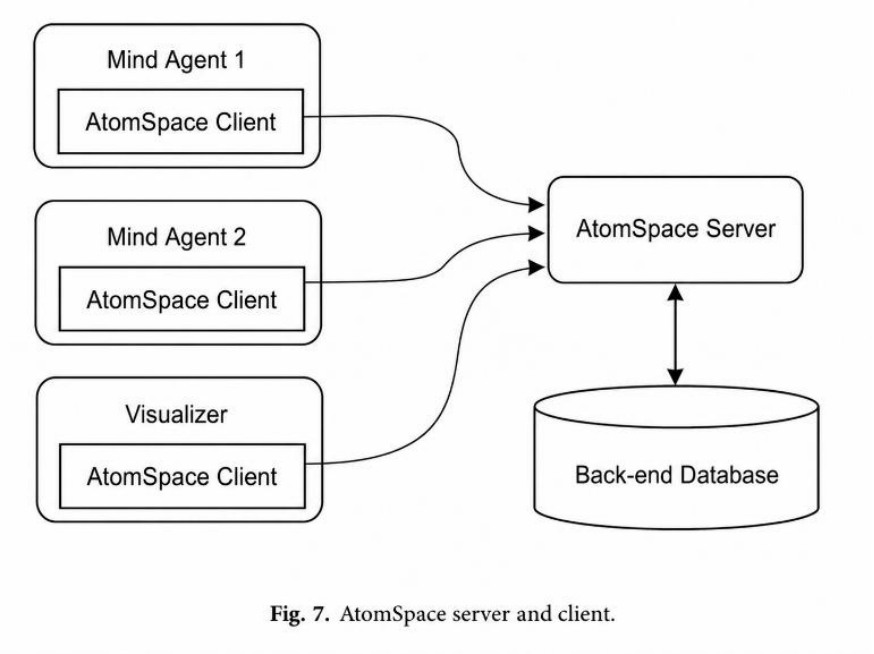


Fig. 7. AtomSpace server and client.

1. Highly distributed. Any component can run *anywhere*, different threads, different processes, even different physical machines, as long as its AtomSpace Client can reach the AtomSpace Server. The back-end database can *also* be spread across machines. Their example: if you use **Redis** (a fast in-memory NoSQL database), the data can be split across several machines, and the AtomSpace Server handles managing them and **load balancing** (spreading the work evenly). So the whole system can scale out horizontally without redesigning it.

2. Parallelism and concurrency. Because each mind agent is simple, has little *direct* relevance to the others, and only talks to the AtomSpace Server, the agents can run **concurrently or in parallel** with no trouble. Then comes the important clarification (the same point from before, restated as a warning): little *direct* communication does **not** mean the agents are independent. They're actually tightly related, because their information exchange happens **implicitly through the shared global knowledge base**. One agent writes Atoms, another reads them. The coupling is through the data, not through conversation. This is the line that prevents a reader from mistaking "decoupled" for "isolated."

3. Cross platform and programming languages. This is a real strength of the design. Once you've ported the **AtomSpace Client** to a given language and platform, you can write a mind agent in *that* language and run it on *that* OS. Their actual implementation proves it, three agents in three different languages all working together:

I. **ActionSelectionMindAgent** in Scheme,

II. **PerceptionUpdaterMindAgent** in C++,

III. **MonitorChangesMindAgent** in Python.

And porting the client to a new language isn't hard, the main work is just writing a **message interpreter** (something that speaks the request-response protocol the server and client use). So the language-agnosticism comes from the fact that everyone communicates through a common message protocol.

***4. Facilitate designing algorithms*.** Because all knowledge sits in one **unified representation** (the hypergraph of Atoms), it's convenient to run **data mining or machine learning** algorithms directly on the knowledge base. It also makes it possible to write **more generic algorithms** that work regardless of where the knowledge came from. Uniform format means tools don't have to be custom-built for each kind of data.

The thread connecting all four: every benefit traces back to the same architectural choice, **simple agents talking to a shared store through a common message protocol**. That single decision is what buys distribution (run anywhere), concurrency (run at once), language freedom (any language that speaks the protocol), and algorithmic convenience (one uniform format to operate on).

4.3. Mind Agents In Depth

The OpenPsi is implemented as six ‘‘mind agents’’ in the architecture described in Section 4.1. Each of them is relatively simple and has a specific task, which can hardly induce complex behavior alone. However, as presented in Fig. 8, they are closely

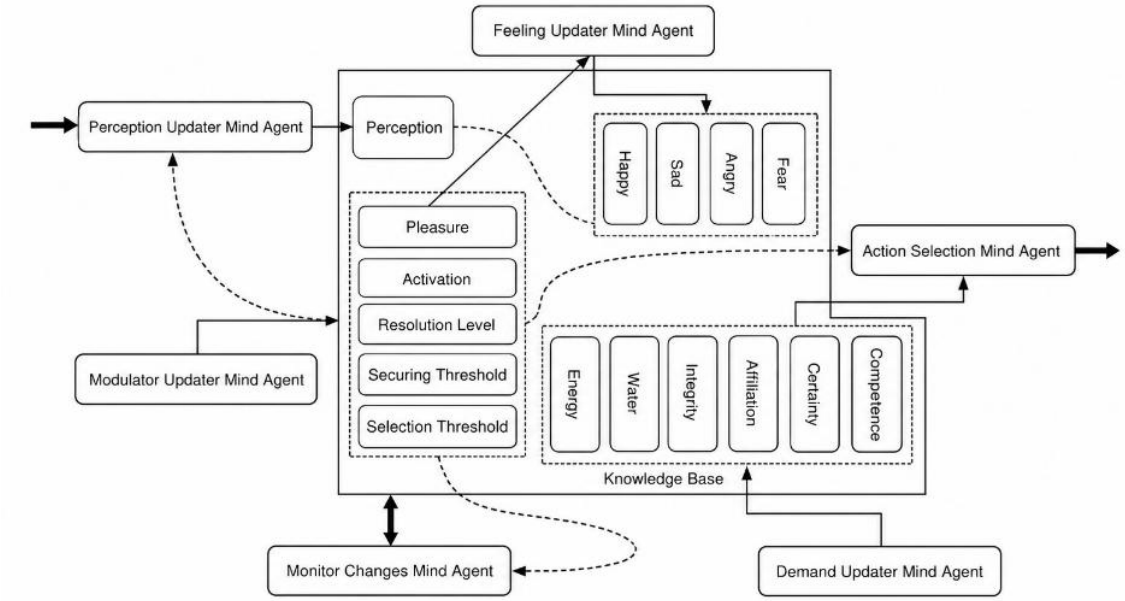


Fig. 8. PSI related mind agents.

related with each other, which would produce much more complicated and interesting behavior. Then the emotions emerge from the dynamics of both internal changes, and the interaction between the system and the environment. Remaining part of this section provides the details of these ‘‘mind agents’’

PerceptionUpdaterMindAgent

keeps the agent's perception of the environment up to date. It's the sensing component, it reads the world and writes what it perceives into the knowledge base.

Perception of the environment the agent's internal picture of what's currently around it in the world (what objects, threats, resources, and other agents it senses), which it builds and refreshes by reading the world.

It's gated by the resolution modulator. This is the key conceptual point and it ties straight back to Section 2. Perception isn't run at full power all the time, it's **modulated by resolution level**. Remember resolution = the accuracy/thoroughness of cognitive processing, and resolution falls as activation rises. So a worked-up, highly activated agent perceives *less carefully*. The modulator literally controls how good the perception is. This is emotion shaping cognition in action, not as a metaphor but as a wired dependency.

"Gated by the resolution modulator" means the resolution modulator acts like a **valve or dial that controls how much perception is allowed to happen**.

The word "gate" comes from the idea of a gate that opens or closes to let something through. So here, resolution level decides *how far open the perception gate is*:

- **High resolution** → gate wide open → the agent perceives carefully, in detail, over a wide area.
- **Low resolution** → gate mostly closed → the agent perceives poorly, in a narrow, sloppy way.

So perception doesn't run at a fixed strength. The resolution modulator regulates it. And since resolution drops when activation rises, an emotionally worked-up agent automatically perceives less. That's the whole point: emotion (via the modulator) controls cognition (perception), rather than the two being separate.

MonitorChangesMindAgent

What it does. The **MonitorChangesMindAgent** watches for **notable changes** in the world. The way it detects them is simple and clever: it **compares the Atoms in the latest AtomSpace against the previous one**. In other words, it looks at the knowledge base now versus a moment ago, and whatever differs is a "change." This is the agent that answers "what just shifted in my environment?"

Just like perception was gated by resolution, change-detection is gated by the **securing threshold** modulator. Recall securing behavior = checking the environment for surprises. The relationship: **lower securing threshold → more frequent change detection**. So a low threshold makes the agent vigilant and constantly scanning; a high threshold makes it check rarely. And remember from Section 2, the threshold drops when the agent is uncertain about its environment, so an unsure agent automatically watches for changes more often. Again, a modulator (emotion-side) directly controls a cognitive process.

What it does with the changes. When it finds significant changes ("mining evident changes" just means extracting the meaningful ones), it **writes those results back into the AtomSpace**. This is important: the changes don't just get reported, they become new Atoms in the shared knowledge base. That's the coordination-through-shared-data principle in action again, this agent communicates its findings simply by writing them where others can read.

How other agents use it. Those change-Atoms act as **heuristic hints** for the other mind agents (a heuristic hint = a useful shortcut/suggestion that guides behavior without being a hard command). The concrete example: they can **bias the PerceptionUpdaterMindAgent** to prioritize the spots where irregular variations occurred. So if something unexpected changed over *there*, perception gets nudged to pay attention *there*. This is a tidy little loop: MonitorChanges spots a surprise → writes it to the AtomSpace → PerceptionUpdater reads it and focuses its attention accordingly.

ActionSelectionMindAgent

The **ActionSelectionMindAgent** is the planning/decision component. It starts from a **dominant demand** (the most pressing need right now) and tries to work out a **chain of actions** that would likely satisfy that demand. So this is the agent that turns "I need energy" into "go to the battery, pick it up, consume it."

Following the same pattern as the other agents, this one is controlled by the **selection threshold** modulator. Recall selection threshold = resistance to switching motives; high threshold means stick with the current goal, low threshold means switch easily. Here it controls how the agent picks which demand (which "active intention") to pursue.

```
IF RANDOM(0,1) < SELECTION_THRESHOLD
  SELECT THE DEMAND WITH LOWEST SATISFACTION
ELSE
  RANDOMLY PICK A DEMAND
```

random(0,1) just draws a random number between 0 and 1. So the logic is a probabilistic switch:

With probability equal to the selection threshold, the agent does the *sensible* thing: it picks the demand with the **lowest satisfaction**, meaning the need that's worst off, the most urgent one.

Otherwise (with probability 1 − selection_threshold), it picks a demand **at random**.

Why design it this way. This is the clever part worth noting in your doc. The selection threshold becomes the agent's **focus/consistency dial**:

High selection threshold → almost always chases the most urgent demand → focused, goal-locked behavior (matches "when fleeing a threat, the agent locks onto survival and won't be distracted").

Low selection threshold → often picks randomly → erratic, easily-distracted, exploratory behavior.

So the random branch isn't a bug, it's deliberately injected noise that lets the agent break out of always doing the obvious thing. It's what allows motives to shuffle and behavior to vary. This is also how the modulator (emotion side) directly shapes decision-making: how concentrated versus scattered the agent's choices are is set by its emotional configuration.

DemandUpdaterMindAgent

What it does. The **DemandUpdaterMindAgent** keeps the agent's **fixed set of demands** updated (energy, integrity, certainty, competence, affiliation). The agent's whole goal in life is to keep these demands **inside healthy ranges**.

Demand satisfaction (eq. 1 and 2)

Each demand `d` has a **minimum level** (`min_l`) and **maximum level** (`max_l`). From the current level `L`, the agent computes a **satisfaction** `S` (a number roughly in [0,1]) using a piecewise rule:

$$S_a(L,\text{min\_l},\text{max\_l},a) = \begin{cases} \text{fuzzy\_equal}(L,\text{min\_l},a) & L < \text{min\_l} \\ 0.9 + 0.1 \cdot \text{rand}(0,1) & L > \text{max\_l} \\ 0.8 + 0.2 \cdot \text{rand}(0,1) & \text{otherwise (inside the band)} \end{cases}$$

Read in plain terms:

- **Below the minimum** → satisfaction decays smoothly via fuzzy_equal (the further below, the lower the satisfaction).
- **Above the maximum** → satisfaction is very high, 0.9 to 1.0, with a little randomness.
- **Inside the band** → satisfaction is high, 0.8 to 1.0, with a bit more randomness.

So being under-supplied is the only thing that hurts; being inside or above the target is fine. The randomness just adds natural jitter.

The fuzzy_equal primitive (eq. 2)

$$\text{fuzzy\_equal}(x, t, a) = \frac{1}{1 + a\,(x - t)^2}$$

This is the single building block the whole model leans on. It's a **bell-shaped curve** that equals 1 when `x = t` (x exactly hits the target) and drops off as x moves away. The parameter `a` controls **how fast** it drops: bigger `a` = sharper, narrower bell; smaller `a` = gentle, wide bell. They use `a = 150` for demands (a fairly sharp bell). `rand(0,1)` just means a random number in [0,1].

The key innovation: target ranges are NOT fixed (eq. 3 and 4)

This is what separates OpenPsi from PSI and MicroPsi, so flag it for your doc. In the original theories, the target ranges are constants. Here they **drift over time** based on experience:

$$\text{min\_l}(t+1) = \begin{cases} b \cdot \text{min\_l}(t) + d & \text{demand was satisfied} \\ b \cdot \text{min\_l}(t) & \text{otherwise} \end{cases}$$

(and the same form for `max_l` in eq. 4)

The idea behind it: when a demand keeps getting satisfied, its target creeps **up** (the agent starts expecting more). When the resource is hard to get, the target drifts **down** (the agent learns to settle for less). This drifting target is how the model produces **personality**, the agent's long-term expectations get shaped by what it has experienced. This connects directly back to Lewis's nested timescales: fast satisfaction events slowly reshape the agent's lasting baseline.

How demand levels are obtained

Two cases:

- **Physiological demands** (energy, integrity) come straight from the **World Interface**, they're read off the environment.
- **Abstract demands** (certainty, competence, affiliation) can't be sensed, so they're **computed** from the agent's experience.

Certainty (eq. 5, 6):

$$L_{\text{certainty}} = \frac{\sum_{i=1}^{\text{object\_num}} \text{fuzzy\_new}(t_i, t_s) + \text{rand}(0,1)}{(1 + e^{-0.05 \cdot \text{object\_num}}) \cdot (\text{object\_num} + 1)}$$

$$\text{fuzzy\_new}(t_i, t_s) = \frac{2}{1 + e^{0.002\,(t_s - t_i)}}$$

In plain terms: certainty is built from how **recently** the agent has seen each known object. `t_i` is the last time object `i` was observed, `t_s` is the current time. The more recently you've seen

things (small `t_s - t_i`), the higher `fuzzy_new`, so recently-confirmed surroundings make the agent feel certain. The denominator just normalizes by the number of objects.

Competence (eq. 7):

$$L_{\text{competence}} = \frac{n_s}{n_s + n_f^{3/2}}$$

`n_s` = number of successful actions, `n_f` = number of failed actions. The clever part is the **`3/2` power on failures**: failures are weighted *more heavily* than successes. So a single failure damages competence more than a single success builds it. That makes the agent's sense of capability cautious and realistic.

Affiliation (eq. 8, 9):

$$L_{\text{affiliation}} = \frac{\sum_{i=1}^{\text{friend\_num}} \text{fuzzy\_near}(d_i, d_{\max}) + \text{rand}(0,1)}{(1 + e^{-0.1 \cdot \text{friend\_num}}) \cdot (\text{friend\_num} + 1)}$$

$$\text{fuzzy\_near}(d_i, d_{\max}) = \frac{1}{1 + 0.00015\,(d_i - d_{\max})}$$

In plain terms: affiliation is built from how **close** friends are. `d_i` is the distance to friend `i`, `d_max` is a cutoff distance that reduces the influence of far-away friends. Nearby friends satisfy the affiliation need; distant ones barely count. (This matches the earlier point: affiliation can only be met by *other agents*.)

ModulatorUpdaterMindAgent

The ModulatorUpdaterMindAgent continuously refreshes the four modulators. Modulators are the parameters that control both cognitive and emotional processes, and because they are closely related to one another, they produce the inherent dynamics of the system. The agent uses the following formulas to update them.

Activation (eq. 10):

$$\text{activation} = \frac{S_{\text{competence}}}{S_{\text{competence}} + 0.5} \cdot \frac{S_{\text{energy}}}{S_{\text{energy}} + 0.05}$$

Activation depends on the energy and competence demands. The avatar becomes more ready to react (higher activation) when it has enough energy and feels confident in its ability.

Resolution level (eq. 11):

$$\text{resolution\_level} = 1 - \sqrt{\text{activation}}$$

Resolution is mainly the reverse of activation. When the avatar is more ready to act (higher activation), it puts less effort into cognitive processes such as perception, change detection, and planning (lower resolution).

Securing threshold (eq. 12):

$$\text{securing\_threshold} = \frac{S_{\text{certainty}}}{S_{\text{certainty}} + 0.05} \cdot S_{\text{integrity}}^{3}$$

Securing threshold depends on certainty and integrity. When the avatar knows more about its environment (higher certainty) or is safe in it (higher integrity), it performs more securing behavior (lower securing threshold).

Selection threshold (eq. 13):

$$\text{selection\_threshold} = \text{fuzzy\_equal}(S_{\text{competence}}, 1{,}15)$$

Selection threshold is tied to competence. When the avatar feels confident in its ability (higher competence), it is less likely to abandon its current action or plan (higher selection threshold).

FeelingUpdaterMindAgent

The FeelingUpdaterMindAgent updates the avatar's emotional states based on pleasure and the modulators. Since the system dynamics are formed by these modulators, the emotional variations derived from them naturally embody the dynamics of the whole system.

The intensity I of each emotion E is computed from the modulator levels m:

$$I(E) = \sum_{m,\, i_m \neq U} W_m \cdot P(m, i_m), i_m \in \{H, L, M, EL, EH, U\}$$

Here W_m is the weight of each modulator's contribution, and P is a fuzzy logic function returning a value in [0,1] based on the modulator level m and its target indicator i_m. The indicator can be H (high), L (low), M (medium), EL (extremely low), EH (extremely high), or U (undefined).

The function P is defined as:

$$P(m, i_m) = \begin{cases} \text{fuzzy\_low}(m, 0.15, 50) & i_m = EL \\ \text{fuzzy\_low}(m, 0.30, 50) & i_m = L \\ \text{fuzzy\_equal}(m, 0.50, 50) & i_m = M \\ \text{fuzzy\_high}(m, 0.70, 50) & i_m = H \\ \text{fuzzy\_high}(m, 0.85, 50) & i_m = EH \\ 0 & i_m = U \end{cases}$$

where the one-sided fuzzy functions are:

$$\text{fuzzy\_low}(x, t, a) = \begin{cases} \text{fuzzy\_equal}(x, t, a) & x > t \\ 1 & \text{otherwise} \end{cases} \text{fuzzy\_high}(x, t, a)$$
$$= \begin{cases} \text{fuzzy\_equal}(x, t, a) & x < t \\ 1 & \text{otherwise} \end{cases}$$

The parameter a has the same meaning as in fuzzy_equal.

After computing the intensities of all possible emotions, the dominant emotion (the one with the greatest intensity) is selected and can be processed further, for example as an emotional expression. A threshold γ is also applied to filter out small emotional fluctuations.

Worked example. Given the relationship between emotions and modulators (Table 2) and the current modulator levels (Table 3), the intensity of each emotion is calculated. Assuming all weights W_m are equal (each 1/5), the intensity of Happy is:

$$I(\text{Happy}) = 0.2\,P(0.4247, H) + 0.2\,P(0.3466, L) + 0.2\,P(0.8436, U) + 0.2\,P(0.7752, H) + 0.2\,P(0.7245, H) = 0.7088$$

With an intensity threshold of γ = 0.6, Happy is selected as the dominant emotion, since its intensity (0.7088) is the highest and exceeds the threshold.

Table 2
Relationship between emotions and modulators. (H, high; L, low; M, medium; EL, extremely low; EH, extremely high; U, undefined.)

Modulator	Happy	Sad	Angry	Fear
Activation	H	L	H	EL
Resolution	L	H	L	EH
Securing threshold	U	U	U	L
Selection threshold	H	EL	L	U
Pleasure	H	EL	EL	EL

Table 3
Current modulator levels.

Modulator	Value
Activation	0.4247
Resolution	0.3466
Securing threshold	0.8436
Selection threshold	0.7752
Pleasure	0.7245

5. Computationalexperiment

5.1. Experimentalsetup

To verify the OpenPsi model, the authors ran experiments in a virtual world full of building blocks, inspired by Minecraft. A virtual robot controlled by OpenPsi lives in this world. It can move freely, and its main mission is to collect enough energy cubes containing batteries to survive.

The robot has the following demands:

I. **Energy level**, which decreases as the robot performs actions and is replenished by consuming battery cubes.
II. **Integrity level**, which decreases when the robot is hurt by a harmful lightning cloud and increases while it stays at home.
III. **Abstract demands**, including certainty and competence.

The robot can also use the building blocks to reach its goals. For example, it can build a staircase to reach battery cubes positioned high above the ground.

The emotional model was then explored across three different scenarios.

DEMAND	DECREASES WHEN	INCREASES WHEN
ENERGY	PERFORMING ACTIONS	CONSUMING BATTERY CUBES
INTEGRITY	HURT BY LIGHTNING CLOUD	STAYING AT HOME
CERTAINTY / COMPETENCE	(COMPUTED FROM EXPERIENCE)	(COMPUTED FROM EXPERIENCE)

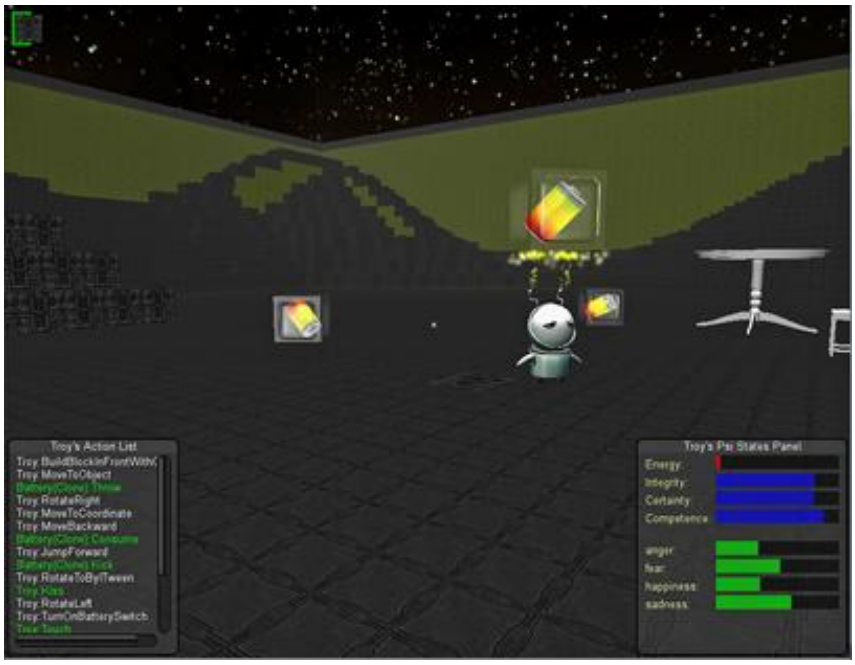


Table 5
Values of variables related to demands and modulators.

Variable	Fixed value	Initial value
Activation	NA	0.85
Resolution	NA	0.80
Securing threshold	NA	0.60
Selection threshold	NA	0.75
Energy level	NA	0.85
Minimum energy level	0.85	NA
Maximum energy level	1.00	NA
Integrity level	NA	0.85
Minimum integrity level	0.80	NA
Maximum integrity level	1.00	NA
Certainty level	NA	0.50
Minimum certainty level	0.70	NA
Maximum certainty level	1.00	NA
Competence level	NA	0.60
Minimum competence level	0.80	NA
Maximum competence level	1.00	NA

The Three Scenarios

Scenario A. The robot is placed in a world with no battery cubes. When it runs out of energy, the human player spawns some energy cubes for it.

Scenario B. The robot is loaded into a world that already contains battery cubes. However, the human player prevents the robot from consuming them, either by throwing the energy cube away or destroying it whenever the robot approaches.

Scenario C. Energy cubes are placed near the lightning cloud. When the robot tries to fetch these batteries, it gets hurt by the harmful cloud.

5.2. Emotionvariationsindifferentscenarios

Scenario A (Fig. 10): The robot experienced happiness, then sadness, then happiness again. It felt happy when first loaded into the empty world, but its happiness faded as it gradually exhausted its energy between 0 and 120 seconds. At the same time, sadness grew, peaking at 270 seconds. The human player spawned energy cubes around 350 seconds. After the robot consumed some batteries and its energy recovered to the target range, it became happy again between 450 and 500 seconds.

Scenario B (Fig. 11): The robot was placed in a world with battery cubes, but the player blocked it from consuming any of them. Happiness decreased and anger grew before 200 seconds, with anger dominating the system from 200 to 300 seconds. After 300 seconds, as the robot ran out of energy, sadness became the major feeling. Anger dropped slightly and held around 0.5, while fear gradually developed and reached 0.6 by 450 seconds.

Scenario C (Fig. 12): Fear increased between 40 and 110 seconds as the robot was hurt by the lightning cloud while trying to fetch energy cubes near it. Sadness also rose as the fear developed, and took over the whole system after 120 seconds.

Scenario	Situation	Emotional trajectory
A	No batteries, then player provides them	Happy → Sad (peak ~270s) → Happy again (450-500s)
B	Batteries present, but player blocks access	Happy → Anger (200-300s) → Sadness, with Fear rising to 0.6
C	Batteries near the harmful lightning cloud	Fear spikes (40-110s) → Sadness takes over (after 120s)

5.3. Differentphasesofemotionaldynamics

Looking at the results more closely, the three phases from Lewis's dynamic theory of emotion can be identified in the simulations.

In **Scenario A** (Fig. 10), the full cycle plays out:

- **Trigger phase (80 to 250s):** small, frequent emotional variations occur, but no single emotion dominates. The system becomes sensitive to the environment and loses its orderliness due to perturbation.
- **Self-amplification phase (250 to 270s):** sadness increases dramatically as small deviations accumulated in earlier stages are rapidly amplified.
- **Self-stabilization phase (after 270s):** sadness decreases as negative feedback gradually takes control of the system dynamics.
- **A second trigger phase (370 to 450s):** the dominant sadness is replaced by many small, frequent fluctuations.
- **A second self-amplification phase (around 450s):** when this trigger phase ends, happiness soars dramatically and becomes the new dominant emotion.

Phase transitions also appear in **Scenarios B and C**, but with a difference. Their trigger stages occur during 200 to 300s and 40 to 70s respectively. Unlike Scenario A, where orderliness is completely lost during the trigger phase, in B and C there is always one emotion slightly stronger than the others throughout the trigger stage: anger is dominant in Scenario B, and fear is dominant in Scenario C.

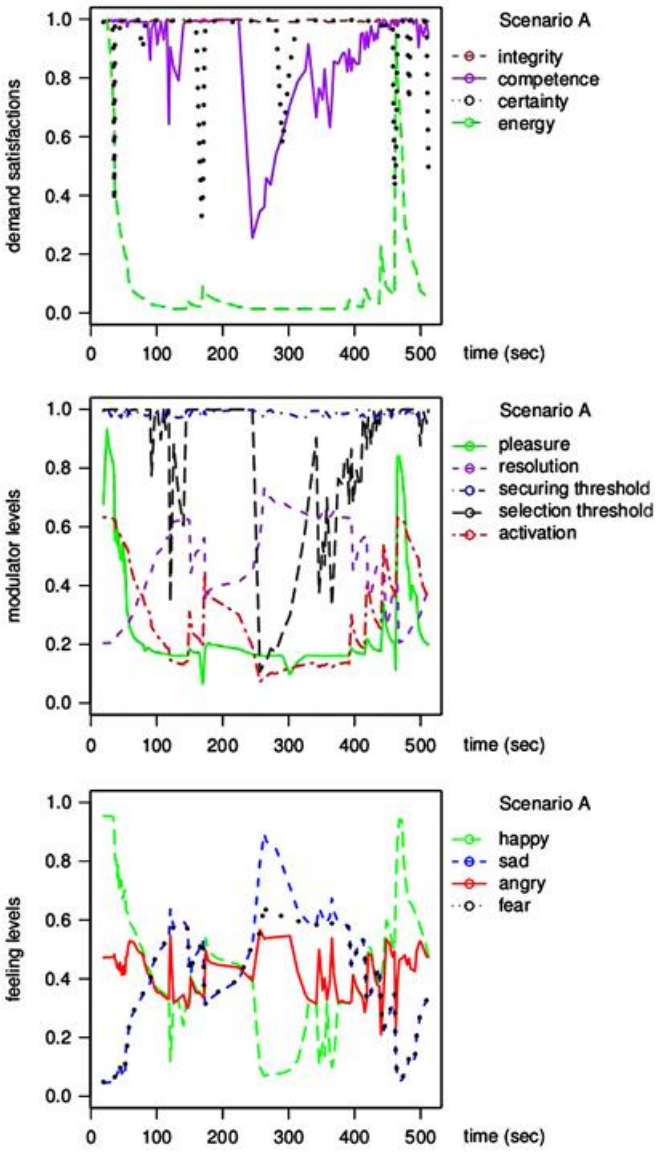


Fig. 10. Dynamics of demands, modulators and feelings in scenario A.

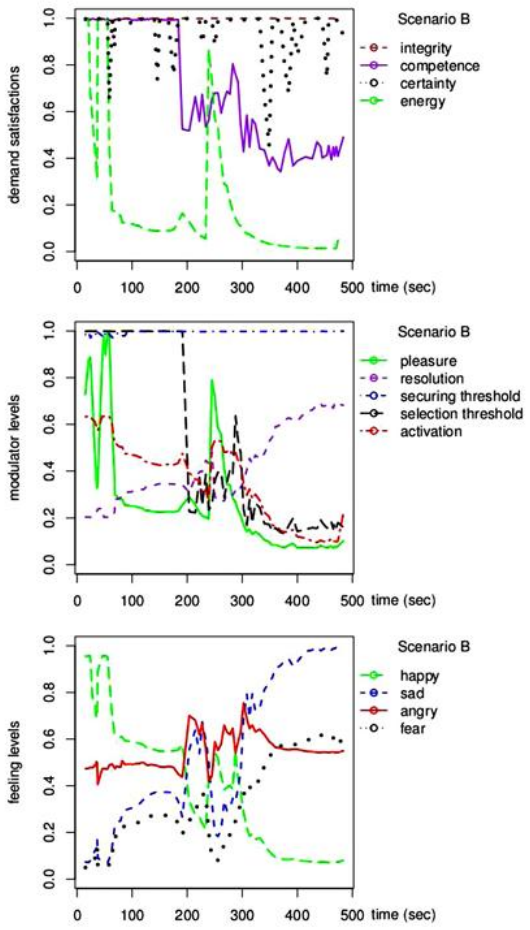


Fig. 11. Dynamics of demands, modulators and feelings in scenario B.

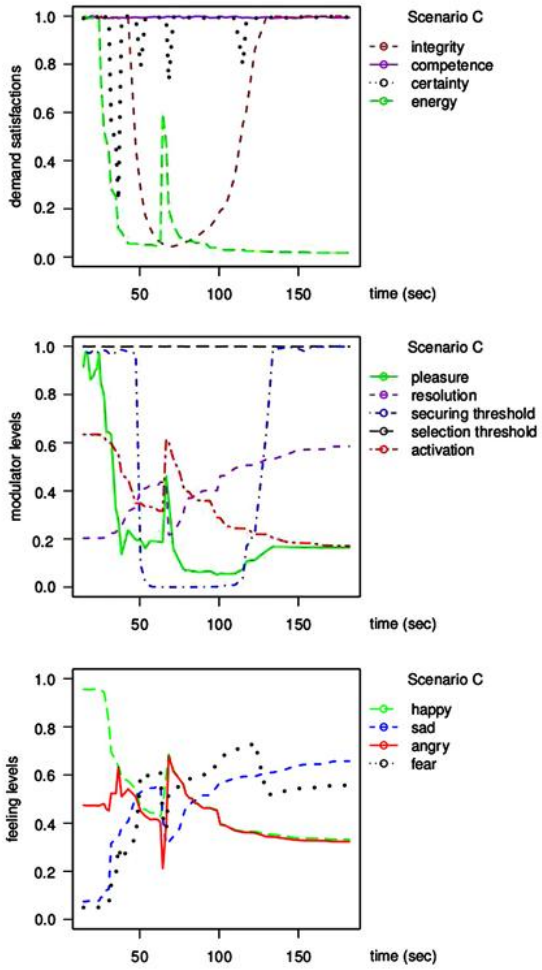


Fig. 12. Dynamics of demands, modulators and feelings in scenario C.

Table 5: Initial values and fixed target ranges

Variable	Initial value	Min (fixed)	Max (fixed)
Activation	0.85	—	—
Resolution	0.80	—	—
Securing threshold	0.60	—	—
Selection threshold	0.75	—	—
Energy level	0.85	0.85	1.00
Integrity level	0.85	0.80	1.00
Certainty level	0.50	0.70	1.00
Competence level	0.60	0.80	1.00

6. Conclusion

This paper introduced OpenPsi, a novel emotional model inspired by Dorner's PSI theory, and constructed and verified it in a game world.

The model was explored across three different scenarios, and the results show that the robot's emotions fit each situation well. The robot became angry when the player kept it from fetching batteries (Scenario B), and afraid when it was hurt by the lightning cloud (Scenario C). The simulations also showed clear evidence of Lewis's phase transitions, including the trigger, self-amplification, and self-stabilization phases.

Importantly, a single fixed set of parameters was used across all experiments. This means the emotion dynamics were produced entirely by the different environmental settings and the interactions between the human player and the robot, not by tuning the model for each case.

These results suggest OpenPsi is a promising approach to emotion modeling, demonstrating three essential features:

1. **Emergent emotions.** Emotions emerge from the dynamics of the whole system, rather than being produced by a set of predefined appraisal rules.
2. **Phase transitions.** Emotion variations go through critical phase transitions, including trigger, self-amplification, and self-stabilization stages.
3. **Environmental grounding.** Emotions are grounded in the environment, so emotional changes fit well with different environmental settings and interactions.

Compared with Joscha Bach's MicroPsi, an extensive implementation of the original PSI theory, OpenPsi focuses only on the most distinctive features of PSI's emotional model. Both MicroPsi and OpenPsi depend on other cognitive components, such as knowledge representation, perception, planning, and learning, because emotion in PSI theory is not an isolated component but emergent behavior of the whole system. The difference lies in how these supporting components are implemented: MicroPsi stays close to the original PSI theory, while OpenPsi uses a completely different knowledge representation scheme, which leads to different planning and learning processes. The authors deliberately implement only the essential motive-driven core of PSI, keeping the flexibility to incorporate state-of-the-art AI algorithms for the remaining components.